

프로젝트 보고서 - 기말

날짜: 2026-06-17, 학번: 214466, 이름: 김건희

1. 프로젝트 개요

본 프로젝트의 이름은 `SpamCheck`이다. FastAPI 기반의 스팸 판별 웹 서비스를 만들고, 기존 키워드 방식 분류기를 ML 모델 기반으로 확장한 뒤 GitHub Actions, Docker, MLflow를 연결하는 것이 목적이다.

프로젝트에서 구현한 내용은 다음과 같다.

- 사용자가 문장을 입력하면 스팸 또는 정상 문장으로 분류한다.
- 학습된 모델 파일을 서비스에서 불러와 예측에 사용한다.
- GitHub Actions에서 테스트와 Docker 빌드 검증을 수행한다.
- MLflow를 이용하여 모델 학습 결과와 평가 지표를 기록한다.
- 모델 버전을 관리하고, 필요할 때 이전 버전으로 되돌릴 수 있도록 스크립트를 작성하였다.

GitHub 저장소 주소:

```
https://github.com/logan0741/gunhee_homework
```

배포 주소:

```
https://gunhee-homework.onrender.com
```

MLflow는 로컬 또는 GitHub Actions 실행 환경에서 다음 주소로 실행하여 확인하였다.

```
cd "2week/1_inital SpamCheck"
python -m mlflow server --host 127.0.0.1 --port 5001
```

서비스 화면과 모델 관리 화면은 아래 캡처로 확인하였다.

AI 스팸 판별기

Win a FREE prize now

분류하기

모델: ml-v2

SpamCheck 서비스 화면

mlflow2.17.2

ExperimentsModels

Experiments

Search Experiments

☐ Default

☒ spamcheck-ml

☐ spamcheck-auto-train

spamcheck-ml

Provide Feedback

Add Description

Share

RunsEvaluationExperimentalTracesExperimental

metrics.rmse < 1 and params.model = "tree"

Time created

State: Active

Datasets

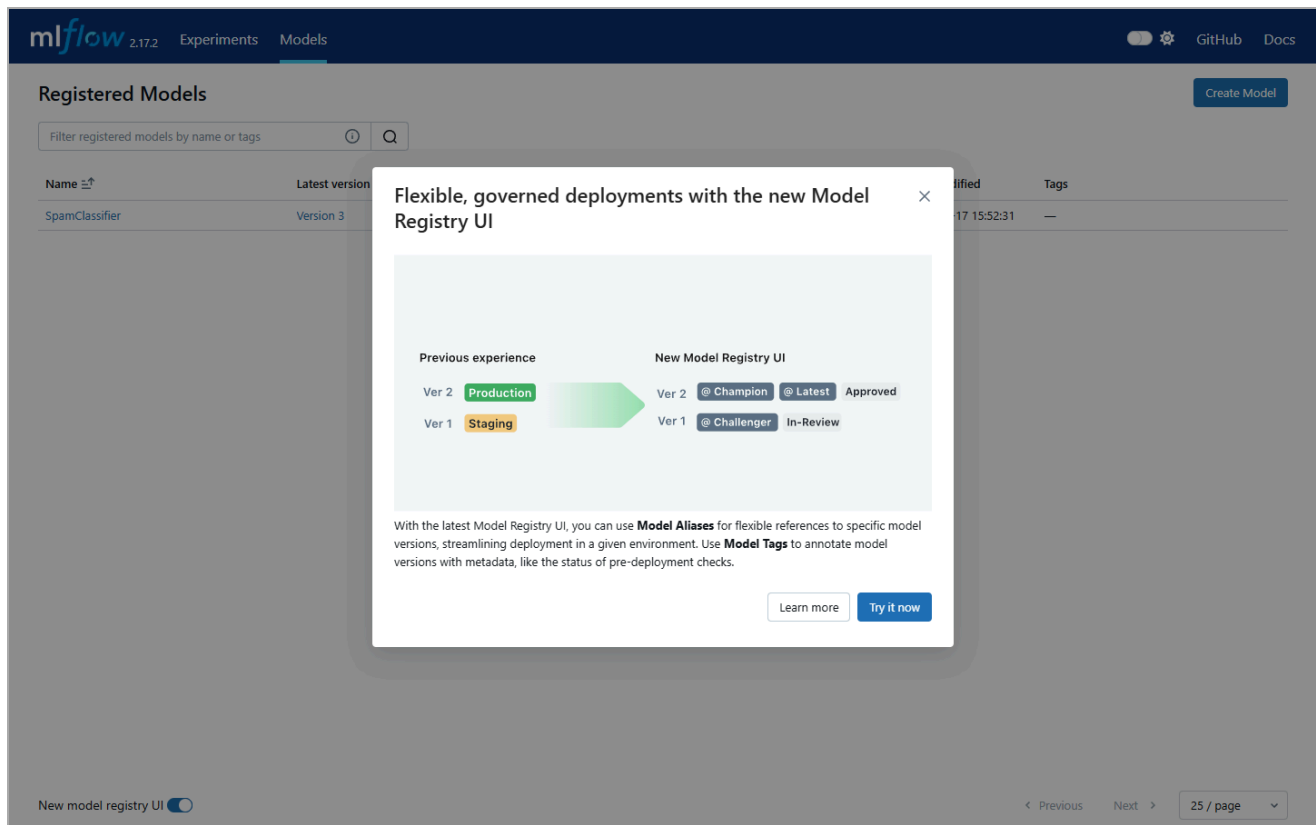
New run

Sort: CreatedColumnsGroup by

	Run Name	Created	Dataset	Duration	Source	Models
☐	skillful-hen-254	15 hours ago	-	8.6s	train.py	SpamClassifier v3
☐	wise-snipe-62	15 hours ago	-	8.9s	train.py	SpamClassifier v2
☐	judicious-sow-73	15 hours ago	-	15.0s	train.py	SpamClassifier v1

3 matching runs

MLflow 실험 목록



MLflow 모델 목록

2. 소프트웨어 주요 기능

사용자가 이용할 수 있는 기능은 크게 네 가지이다.

기능	설명
스팸 판별	입력 문장을 <code>spam</code> 또는 <code>ham</code> 으로 분류한다.
예측 점수 표시	모델이 계산한 스팸 확률을 함께 반환한다.
모델 정보 확인	<code>/model/info</code> 에서 현재 사용 중인 모델 상태를 확인한다.
모델 다시 불러오기	<code>/model/reload</code> 에서 서버 실행 중 모델 파일을 다시 읽어 온다.

ML 기능은 `app/spam.py`에 분리하였다. 서비스 요청 처리는 `app/main.py`에서 담당하고, 학습은 `train.py`에서 수행한다. 이 구조로 서비스 코드와 모델 학습 코드를 구분하였다.

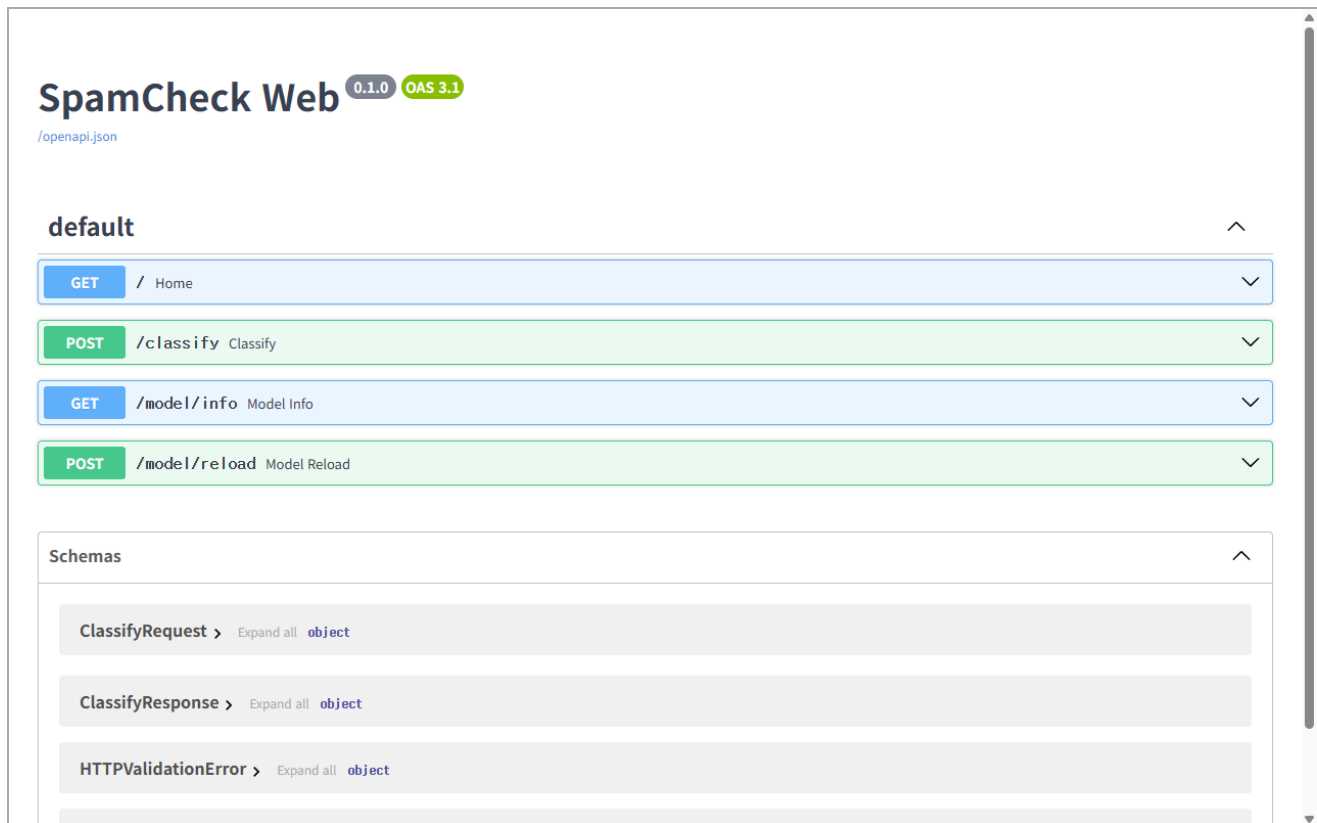
입력 예시는 다음과 같다.

```
{
  "text": "You have won a FREE prize click here now"
}
```

출력 예시는 다음과 같다.

```
{
  "label": "spam",
  "score": 0.7811
}
```

Swagger 화면에서도 API 구조를 확인할 수 있다.



FastAPI Swagger 화면

3. 실행 환경

항목	내용
사용 OS	Windows 11
로컬 Python	Python 3.8 계열
Docker Python	Python 3.10-slim
주요 프레임워크	FastAPI, scikit-learn, MLflow
테스트 도구	pytest, httpx

배포 환경	Render Docker 기반 배포
저장소	logan0741/gunhee_homework

의존성은 `requirements.txt`에 정리하였다. Docker 환경에서는 같은 의존성을 설치한 뒤 `uvicorn`으로 서비스를 실행하도록 하였다.

4. 전체 MLOps 파이프라인 구조

이번 프로젝트의 전체 흐름은 다음과 같이 구성하였다.

코드 수정

- > `git commit`
- > `git push`
- > GitHub Actions 테스트 실행
- > Docker 이미지 빌드 확인
- > `train.py`로 모델 학습
- > MLflow에 파라미터와 지표 기록
- > `models/spam_pipeline.pkl` 저장
- > 서비스에서 모델 파일 로드

코드 변경 흐름은 일반적인 CI 흐름과 같다. 로컬에서 코드를 수정한 뒤 push하면 GitHub Actions가 테스트를 실행한다. 테스트가 통과하면 Docker 이미지 빌드와 컨테이너 실행도 확인한다.

모델 학습 흐름은 `train.py`를 중심으로 구성하였다. 데이터셋을 불러오고, TF-IDF와 Naive Bayes 모델을 학습한 뒤 결과를 MLflow에 남긴다.

모델 반영은 `models/spam_pipeline.pkl` 파일을 기준으로 한다. 서비스가 시작될 때 해당 파일을 읽고, `/model/reload`를 호출하면 실행 중에도 다시 불러올 수 있다.

5. Git 기반 개발 과정

초기에는 키워드 기반 스팸 판별 기능을 먼저 만들었다. 이후 테스트, Docker, GitHub Actions, 로깅, MLflow 학습 기능을 순서대로 추가하였다.

주요 개발 순서는 다음과 같다.

1. FastAPI 기본 앱 구성
2. 키워드 기반 스팸 판별 함수 작성
3. 테스트 코드 추가
4. Dockerfile 및 compose 설정

5. GitHub Actions workflow 작성
6. 로깅 및 오류 처리 추가
7. TF-IDF + Naive Bayes 모델 학습 코드 작성
8. MLflow 기록, 모델 승격, 롤백 스크립트 추가

커밋 메시지는 기능 추가, 수정, 문서 작업을 구분하려고 하였다.

구분	예시
기능 추가	feat: upgrade to ML model
오류 수정	fix: Python 3.8 compatibility
설정 변경	chore: update trained model
문서 작성	docs: add final project report

실제 커밋 이력 일부는 다음과 같다.

```
10fad47 fix: Python 3.8 compatibility
e58c084 fix: MLFlow server inside Actions runner
edfc688 Add MLFlow auto train workflow
c2c4e4d Add logging and GitHub issue reporting
13047e4 Dockerize for deployment
```

git log --oneline (C:\W...SpamCheck)

```
4aa787d feat: upgrade to ML model (TF-IDF + Naive Bayes) with full MLOps pipeline
10fad47 fix: add from __future__ import annotations for Python 3.8 compatibility
e58c084 fix: run MLFlow server inside Actions runner to avoid ngrok host header issue
edfc688 Add MLFlow auto train workflow
c2c4e4d Add logging and GitHub issue reporting
13047e4 Dockerize for deployment
4c3c212 Merge pull request #3 from logan0741/fix/issue-3-swagger-docs
6639583 fix: add request schema for classify docs
8e181db Merge pull request #1 from logan0741/feature/ui-title
8c57c34 ui: improve title visibility
cf8799e Merge branch 'feature/ui-soomi'
b6758c9 ui: clarify title with AI keyword
2e58588 ui: translate title
e0ac00c feature: extend spam keywords
787bfa1 resolve: merge UI title changes
ae935e5 ui: clarify title with AI keyword
```

Git 커밋 이력

6. CI/CD 구성

GitHub Actions workflow는 네 개를 사용하였다.

workflow	역할
ci.yml	pytest 실행
docker-verify.yml	Docker 이미지 빌드와 컨테이너 실행 확인
mlflow-train.yml	모델 학습 및 MLflow 기록
auto-promote.yml	정확도 기준으로 모델 stage 변경

ci.yml에서는 Python 환경을 만든 뒤 requirements.txt를 설치하고 pytest를 실행한다. 현재 로컬 기준으로 테스트는 9개가 통과하였다.

9 passed

docker-verify.yml에서는 Docker 이미지를 만들고 컨테이너를 실행한 뒤 HTTP 응답을 확인한다. 배포 전에 컨테이너 실행 문제가 있는지 확인하기 위한 과정이다.

mlflow-train.yml에서는 runner 내부에서 MLflow 서버를 실행한 뒤 train.py를 실행한다. 외부 서버 연결 문제를 피하기 위해 Actions 내부의 localhost 서버를 사용하도록 수정하였다.

GitHub Actions 실행 화면

7. Docker 기반 환경 구성

서비스 Dockerfile은 Python 3.10 slim 이미지를 사용한다.

```
FROM python:3.10-slim

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
ENV PORT=10000

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 10000

CMD ["sh", "-c", "uvicorn app.main:app --host 0.0.0.0 --port ${PORT}"]
```

주요 설정 이유는 다음과 같다.

- `python:3.10-slim` 을 사용하여 실행 환경을 고정하였다.
- `requirements.txt` 를 먼저 복사하여 Docker 레이어 캐시를 활용하였다.
- Render에서 사용할 수 있도록 `PORT` 환경 변수를 사용하였다.
- 로그가 바로 출력되도록 `PYTHONUNBUFFERED=1` 을 설정하였다.

MLflow 서버용 Dockerfile도 별도로 작성하였다.

```
FROM python:3.10-slim
RUN pip install --no-cache-dir mlflow==2.22.0
WORKDIR /mlflow
EXPOSE 5000
CMD ["mlflow", "server", "--host", "0.0.0.0", "--port", "5000"]
```

로컬 실행 방법은 다음과 같다.

```
docker build -t spamcheck:local .
docker run --rm -p 10000:10000 -e PORT=10000 spamcheck:local
```

8. ML 모델 구성

처음에는 키워드 방식으로 스팸을 판별하였다. 기말 프로젝트에서는 이를 ML 모델로 변경하였다.

사용한 데이터는 `data/dataset.py`에 있는 직접 작성 데이터이다.

- 전체 데이터: 100개
- 스팸 데이터: 50개
- 정상 데이터: 50개
- 학습/테스트 분할: 80:20

모델은 scikit-learn Pipeline으로 구성하였다.

```
Pipeline([
    ("tfidf", TfidfVectorizer(
        max_features=1000,
        ngram_range=(1, 2),
        stop_words="english",
    )),
    ("clf", MultinomialNB(alpha=1.0)),
])
```

학습 과정은 다음과 같다.

1. 데이터셋을 불러온다.
2. 학습 데이터와 테스트 데이터를 나눈다.
3. TF-IDF로 문장을 벡터화한다.
4. MultinomialNB 모델을 학습한다.
5. accuracy, precision, recall, f1을 계산한다.
6. 결과를 MLflow에 기록한다.
7. 학습된 모델을 `models/spam_pipeline.pkl`로 저장한다.

키워드 방식과 ML 방식의 비교 결과는 다음과 같다.

구분	키워드 방식	ML 방식
방식	지정 키워드 개수 확인	TF-IDF + Naive Bayes
Accuracy	0.8500	1.0000
F1	0.8235	1.0000
장점	단순하고 빠름	키워드 목록에 없는 표현도 일부 대응 가능
한계	키워드 목록에 의존	데이터가 적으면 일반화 성능을 판단하기 어려움

현재 데이터셋은 크기가 작기 때문에 100% 정확도를 실제 운영 성능으로 보기에는 한계가 있다. 이 수치는 과제용 데이터셋의 테스트 분할에서 나온 결과로 정리하였다.

8. MLflow 기반 실험 관리

MLflow 실험 이름은 `spamcheck-ml`로 설정하였다. `train.py`에서 모델 파라미터, 평가 지표, 모델 아티팩트를 기록하도록 하였다.

기록한 파라미터는 다음과 같다.

항목	값
model_type	TF-IDF + MultinomialNB
max_features	1000
ngram_range	(1,2)
nb_alpha	1.0
train_samples	80
test_samples	20

기록한 지표는 다음과 같다.

지표	값
v2_accuracy	1.0000
v2_precision	1.0000
v2_recall	1.0000
v2_f1	1.0000
v1_accuracy	0.8500
v1_f1	0.8235

MLflow에는 `spam_pipeline` 아티팩트와 `SpamClassifier` 모델 버전이 기록된다.

The screenshot shows the MLflow Run page for a specific run named 'skillful-hen-254'. The interface includes a top navigation bar with 'mlflow 2.17.2', 'Experiments', and 'Models' tabs. The main content area is divided into sections: 'Description' (No description), 'Details' (a table of run metadata), 'Parameters (6)', and 'Metrics (8)'.

Parameter	Value
max_features	1000
model_type	TF-IDF + MultinomialNB

Metric	Value
v1_accuracy	0.85
v1_f1	0.8235294117647058

MLflow Run 상세 화면

The screenshot shows the MLflow Model page for a model named 'SpamClassifier'. The page displays the model's version history in a table. A modal dialog is open in the center, titled 'Flexible, governed deployments with the new Model Registry UI', which compares the 'Previous experience' with the 'New Model Registry UI'.

Version	Registered at
Version 3	2026-06-17 15:45:48
Version 2	2026-06-17 15:45:19
Version 1	2026-06-17 15:40:32

MLflow 모델 상세 화면

9. 모델 등록 및 서비스 반영

모델은 두 가지 방식으로 관리하였다.

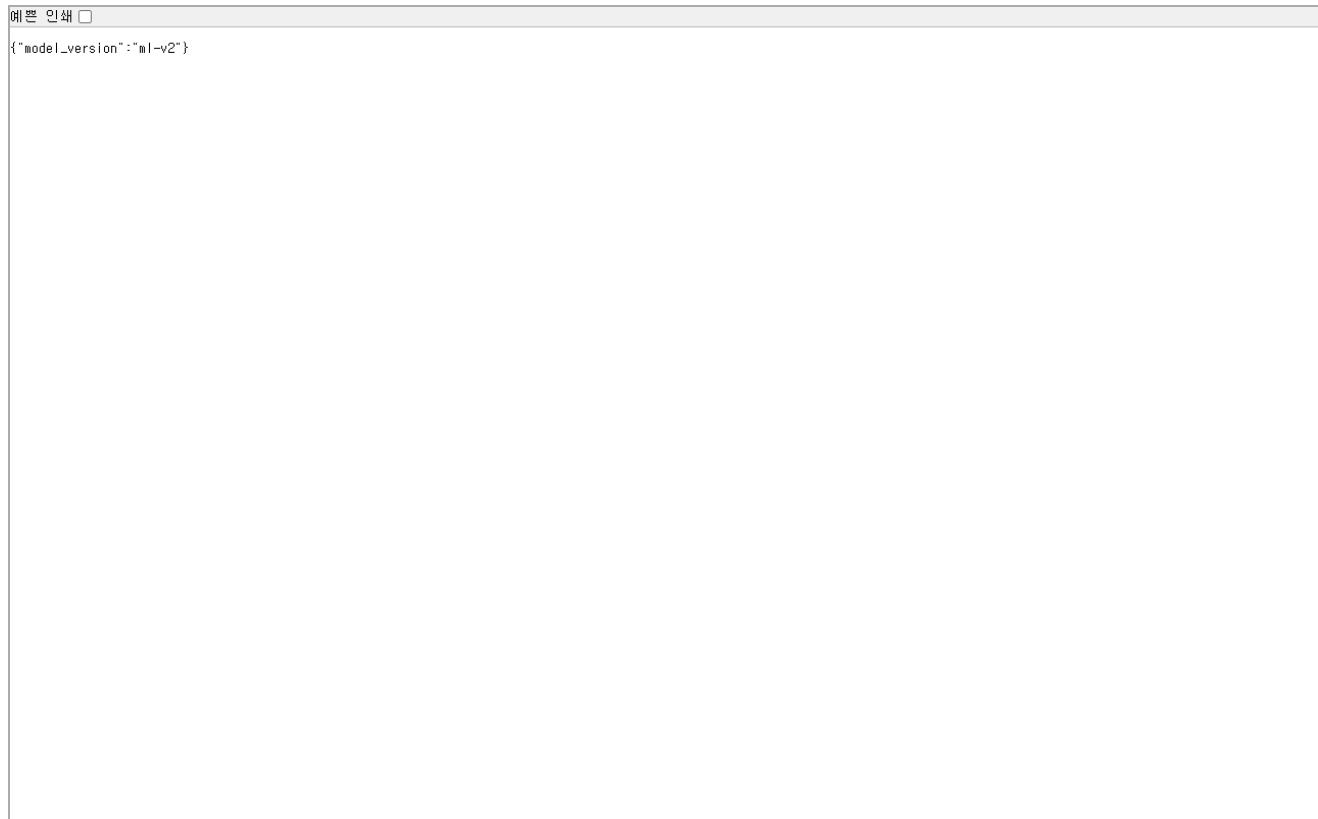
첫째, MLflow Model Registry에 `SpamClassifier`라는 이름으로 등록하였다. 이 방식은 실험별 모델 버전을 남기기 위한 것이다.

둘째, 서비스에서 바로 사용할 수 있도록 `models/spam_pipeline.pkl` 파일로 저장하였다. FastAPI 서비스는 이 pickle 파일을 읽어 예측에 사용한다.

서비스에서 모델을 불러오는 방식은 다음과 같다.

```
model_path = Path(__file__).resolve().parents[1] / "models" / "spam_pipeline.pkl"
if model_path.exists():
    with open(model_path, "rb") as f:
        _MODEL = pickle.load(f)
        _MODEL_VERSION = "ml-v2"
else:
    _MODEL_VERSION = "keyword-v1"
```

모델 파일이 있으면 ML 모델을 사용하고, 없으면 키워드 방식으로 동작한다. `/model/info`에서 현재 상태를 확인할 수 있다.



모델 정보 API

새 모델을 반영하는 방법은 두 가지이다.

1. 모델 파일을 저장소에 반영한 뒤 Render에서 재배포한다.

2. 실행 중인 서비스에서 `/model/reload`를 호출하여 모델 파일을 다시 불러온다.

10. 재학습 또는 모델 개선 과정

모델 개선은 기존 키워드 방식과 ML 방식의 성능 비교를 목적으로 진행하였다.

실험에서 바꾼 내용은 다음과 같다.

실험	max_features	ngram_range	nb_alpha	설명
1	1000	(1,2)	1.0	기본 설정
2	200	(1,1)	2.0	더 단순한 설정
3	1000	(1,2)	1.0	최종 설정 재확인

모든 실험에서 현재 테스트셋 기준으로 ML 방식이 키워드 방식보다 높은 성능을 보였다. 다만 데이터 수가 작기 때문에 실제 서비스 품질을 판단하려면 더 큰 데이터셋이 필요하다.

모델 교체 결과, 서비스에서는 `m1-v2` 모델이 사용되는 것을 확인하였다. 로컬에서 `/model/info`와 `/classify` API를 호출하여 모델이 로드되어 있는지 확인하였다.

```
GET /model/info
{"model_version":"m1-v2"}

POST /classify
입력: You have won a FREE prize click here now
응답: {"label":"spam","score":0.7811}
```

11. 운영 로그 및 문제 대응

`app/main.py`에서는 `/classify` 요청과 처리 결과를 로그로 남긴다.

로그에 포함되는 내용은 다음과 같다.

- 요청한 endpoint
- 입력 문장과 길이
- 모델 버전
- 분류 결과와 점수
- 오류 발생 시 예외 종류와 traceback

오류 상황을 확인하기 위해 입력값이 `crash`일 때 예외가 발생하도록 하였다.

```
if text == "crash":
    raise RuntimeError("의도적 장애 추가")
```

오류가 발생하면 사용자에게는 내부 traceback을 직접 보여주지 않고, API 응답으로 `label: error, score: -1.0`을 반환한다. 또한 `GH_REPO`와 `GH_TOKEN` 환경 변수가 설정되어 있으면 GitHub Issue를 생성하도록 하였다. 환경 변수가 없으면 이슈 생성을 건너뛰고 로그만 남긴다. 이 방식으로 운영 중 오류가 발생했을 때 로그와 이슈를 통해 원인을 확인할 수 있도록 하였다.

12. 롤백 및 이전 모델 관리

이전 모델은 MLflow Model Registry에 버전 단위로 남는다. 각 버전은 stage 값을 가질 수 있다. 예시로 정리한 버전 상태는 다음과 같다.

버전	Stage	설명
v1	Archived	초기 설정 모델
v2	None	단순 파라미터 모델
v3	Production	최종 선택 모델

롤백은 `scripts/rollback.py`로 수행한다.

```
python scripts/rollback.py --list
python scripts/rollback.py --version 1
```

스크립트는 지정한 버전을 Production으로 변경하고, 기존 Production 버전은 Archived로 바꾼다. 실제 서비스 반영은 최종 모델 파일을 다시 저장하거나 배포하는 과정과 함께 진행해야 한다.

13. 전체 파이프라인 동작 흐름

코드 수정 후 서비스 반영까지의 흐름은 다음과 같다.

```
코드 수정
-> git commit
-> git push
-> GitHub Actions 테스트
-> Docker 검증
```

```
-> Render 재배포  
-> 서비스 접속 확인
```

데이터 변경 후 재학습까지의 흐름은 다음과 같다.

```
data/dataset.py 수정  
-> git commit  
-> git push  
-> mlflow-train.yml 실행  
-> train.py 학습  
-> MLflow 기록  
-> 모델 파일 저장
```

모델 변경 후 운영 반영까지의 흐름은 다음과 같다.

```
모델 학습  
-> MLflow에 새 버전 기록  
-> 모델 파일 저장  
-> 서비스 재배포 또는 /model/reload 호출  
-> /model/info로 확인
```

14. 문제 해결 경험

14-1. Python 3.8 타입 힌트 문제

Python 3.8 환경에서 `tuple[str, float]` 형식의 타입 힌트를 사용할 때 문제가 생길 수 있었다. 이를 해결하기 위해 `app/spam.py` 상단에 다음 코드를 추가하였다.

```
from __future__ import annotations
```

이 설정을 통해 타입 힌트가 런타임에 바로 평가되지 않도록 하여 Python 3.8에서도 실행할 수 있도록 하였다.

14-2. MLflow 서버 연결 문제

처음에는 GitHub Actions에서 외부 MLflow 서버에 연결하려고 하였으나 Host header 관련 오류가 발생하였다. 이후 Actions runner 내부에서 MLflow 서버를 직접 실행하고 localhost로 접속하도록 변경하였다.

```
- name: Start MLflow tracking server
  run: |
    mlflow server --host 0.0.0.0 --port 5000 --backend-store-uri sqlite:///mlflow.db &
    sleep 5
```

이 방식으로 Actions 환경에서도 학습 결과를 MLflow에 기록할 수 있었다.

14-3. 테스트 캐시 경고

로컬에서 pytest를 실행했을 때 `.pytest_cache` 경로에 대한 권한 경고가 발생하였다. 테스트 자체는 통과하였고, 기능 실패가 아니라 캐시 파일 생성 실패에 해당하였다. 보고서에는 테스트 결과와 함께 이 점을 구분하여 정리하였다.

15. 느낀 점 및 개선 방향

이번 프로젝트를 통해 모델을 학습하는 것과 서비스를 운영하는 것은 별개의 문제가 아니라는 점을 확인하였다. 모델 파일이 있어도 서비스에서 제대로 불러오지 못하면 사용할 수 없고, 서비스가 실행되어도 테스트와 로그가 없으면 운영 중 문제를 찾기 어렵다.

개선하고 싶은 부분은 다음과 같다.

- 현재 데이터셋은 100개로 작기 때문에 더 큰 공개 데이터셋을 사용해 보고 싶다.
- 현재 모델은 영어 스팸 문장 중심이므로 한국어 스팸 문장도 추가할 필요가 있다.
- MLflow 서버를 별도 서비스로 배포하여 실험 결과를 더 안정적으로 보관하고 싶다.
- 운영 로그를 모아 모델 성능이 떨어지는 시점을 확인하는 구조도 추가할 수 있다.

이번 과제에서는 Git, GitHub Actions, Docker, MLflow가 따로 존재하는 도구가 아니라 하나의 개발 흐름 안에서 연결된다는 점을 확인할 수 있었다.

참고 자료

- 수업 자료: `10_2_MLOps 개요.pdf`
- 수업 자료: `11_1_MLSpamchecker와 MLflow 개요.pdf`
- 수업 자료: `11_2_MLflow 로컬 실습.pdf`
- 수업 자료: `12_1_MLflow 서버 실습.pdf`
- 수업 자료: `12_2_GitHub Actions 기반 자동 훈련.pdf`
- MLflow 공식 문서: <https://mlflow.org/docs/latest/index.html>
- scikit-learn Pipeline 문서: <https://scikit-learn.org/stable/modules/compose.html>
- FastAPI 공식 문서: <https://fastapi.tiangolo.com>

- Render Docker 배포 문서: <https://render.com/docs/docker>